



Software vulnerabilities in TensorFlow-based deep learning applications

Katarzyna Filus*, Joanna Domańska

Institute of Theoretical & Applied Informatics, Polish Academy of Sciences (IITIS-PAN), ul. Baltycka 5, 44100 Gliwice, Poland

ARTICLE INFO

Article history:

Received 31 March 2022

Revised 28 September 2022

Accepted 5 October 2022

Available online 11 October 2022

Keywords:

Software vulnerability

TensorFlow

Deep learning

Security

Static analysis

ABSTRACT

Usage of Deep Learning (DL) methods is ubiquitous. It is common in the DL/Artificial Intelligence domain to use 3rd party software. TensorFlow is one of the most popular Machine Learning (ML) platforms. Every software product is a subject to security failures which often result from software vulnerabilities. In this paper, we focus on threats related to 6 common types of threats in TensorFlow implementation. We identify them using Common Weakness Enumeration. We analyze more than 100 vulnerability instances. We focus on vulnerabilities' severity, impact on confidentiality, integrity and availability, as well as possible results of exploitation. We also use Orthogonal Defect Classification (ODC). The results show that a majority of vulnerabilities are caused by missing/incorrect checking statements, however some fixes require more advanced algorithmic changes. Static Analysis Tools tested in our study show low effectiveness in detecting known vulnerabilities in TensorFlow, but we provide some recommendations based on the obtained alerts to improve overall code quality. Further analysis of vulnerabilities helped us to understand and characterize different vulnerability types and provide a set of observations. We believe that these observations can be useful for the creators of new static analysis tools as a source of inspiration and to build the test cases. We also aim to draw the programmers' attention to the prevalence of vulnerabilities in deep learning applications and a low effectiveness of automatic tools to find software vulnerabilities in such products.

© 2022 The Author(s). Published by Elsevier Ltd.

This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>)

1. Introduction

Artificial Intelligence (AI) and Deep Learning (DL) play a crucial role in digital transformation through automation of decision-making process. Although the benefits of these technologies are significant, so are the concerns. In the domain of AI, new security threats appear. The AI threat landscape is very difficult to map, as it involves many advanced methods and complex pipelines. Also, AI products often use 3rd party platforms. TensorFlow is one of the most popular such platforms (Abu Daoud et al., 2021), or even the most popular (Pang et al., 2020). It is used by the technology giants such as PayPal or eBay (TensorFlow, 2021). The same as for every other software product, it is a subject to security failures often resulting from software vulnerabilities (a flaw resulting from a mistake in software specification, implementation or configuration (Committee et al., 1990)). Under some conditions faulty code execution can impact Confidentiality, Availability, or Integrity (CIA triad) of a system (Corallo et al., 2020).

To improve security, organizations such as MITRE and OWASP provide guidelines (e.g. OWASP Secure Coding Practices Guide (OWASP Secure Coding Practices Quick Reference Guide, 2021)) and conduct standardization activities. The most common standards are MITRE's Common Vulnerabilities and Exposures (CVE) (CVE - Common Vulnerabilities and Exposures, 2021) and Common Weakness Enumeration (CWE) (CWE - Common Weakness Enumeration, 2021). Such standards facilitate weakness identification, mitigation, as well as prevention. Also, rankings are created to raise awareness regarding the most dangerous weaknesses, e.g. CWE TOP25 (CWE Top 25 Most Dangerous Software Weaknesses, 2021). Moreover, it is recommended to use tools, e.g. Static Analysis Tools (SATs) to analyse software and find potential vulnerabilities.

Despite the fact that a lot of work is being done to improve security, vulnerabilities are still extremely common. The most common and dangerous vulnerabilities are those related to C/C++ languages (According to Veracode's report (Veracode, 2020) and CWE TOP25 ranking (CWE Top 25 Most Dangerous Software Weaknesses, 2021)). They offer high control and efficiency, which come with a cost - high severity threats. Due to their efficiency, C and C++ are used to build high-level numerical libraries (e.g. Tensor-

* Corresponding author.

E-mail addresses: kfilus@iitis.pl (K. Filus), joanna@iitis.pl (J. Domańska).

Flow or Caffe). Although there are many other (dedicated) threats in AI applications, it is essential to focus on these known vulnerabilities, as those are often forgotten in the battle against the AI-centred security threats.

Our paper aims to characterize six different CWE types of software vulnerabilities in TensorFlow implementation (104 CVE instances): related to memory management issues, integer overflow, improper input validation and NULL pointer dereference. **To achieve this, our study consists of four complementary directions:**

1. Analysis of metadata available on CVEDetails.com: statistics regarding Common Vulnerability Scoring System (CVSS) scores, impact on CIA triad and results of exploitation.
2. Analysis of vulnerabilities and their fixes in TensorFlow based on commits regarding these fixes with Orthogonal Defect Classification (ODC).
3. Analysis of SAT alerts obtained with CppCheck, FlawFinder and Visual Code Grepper for vulnerable and neutral files (before and after fix).
4. Analysis of descriptions of vulnerabilities and their fixes available on CVE Details.com and TensorFlow github Advisory and code changes.

Our contribution: The outcome of the study is an analytical study regarding the TensorFlow vulnerabilities along with the set of observations and the main findings regarding these vulnerabilities. We describe vulnerabilities at the CWE type level along with examples from the TensorFlow implementation. We have compiled a comprehensive knowledge base regarding main causes of different CWE types, their results and exploitation ways. We also focused on the ways to avoid/minimize the exploitation chance. We analyzed a larger number of CWE types than other works in the literature (they usually analyze one or two CWE types at once). Moreover, our work seems to be the first one analyzing a DL library itself and not its dependencies. We used our main findings to create a checklist. The checklist includes the characteristic for the numerical libraries functionalities and cases, in which there is a high chance of introducing security vulnerabilities. We hope that our work will help the developers of numerical libraries to spot the parts of the code being developed that can become potential security hot-spots. When it comes to developers of static analysis tools - we believe that our insights can be used to create new rules in the rule-based systems, and also to build test cases for such solutions. Also, we could not find any work testing the usefulness of SATs for detecting vulnerabilities in deep learning products, therefore we decided to test 3 freely available SATs. Our concise evaluation (mainly the qualitative one but with some quantitative results) and discussion of different SATs can serve as a guide for selecting free tools to assess code quality. We hope that our paper will raise awareness regarding Tensorflow and related security issues. Although the described vulnerabilities are well-known, they are still a major threat to C/C++ software products, therefore raising awareness of these issues should be an ongoing effort - especially in the domain of AI and Deep Learning, in which algorithms are often treated as miraculous black box machines for solving a plethora of problems.

The rest of the paper is organized as follows: Section 2 explains the term “software vulnerabilities” and describes methods that can be used to detect them. It also shortly outlines the AI vulnerability landscape and introduces a clear division of “high-level” AI-dedicated and “low-level” traditional threats - characteristic for C/C++ products (we focus on these vulnerabilities in our study). We also describe the related works in that section. In Section 3, we describe our approach to the analysis and the methods used in our study. In Section 4, we analyze the metadata regarding different vulnerability types available at CVEDetails.com. In Section 5, we

perform the Orthogonal Defect Classification and describe its results. In Section 6, we use three freely available SATs to analyze all of the code files taken into consideration in our study. Section 7 focuses on the analysis of descriptions, advisories and source code files of known vulnerabilities. Section 8 concludes the paper. In Appendix A, we provide our checklist regarding the security threats and how to avoid them.

2. Background and related work

2.1. Software vulnerabilities

Vulnerability is defined by ISO/IEC 27000:2018 (ISO/IEC 27000:2018, 2018) as a “weakness of an asset or control that can be exploited by one or more threats”. Undetected vulnerabilities ultimately entail significant maintenance costs (Zhioua et al., 2014) and can cause potential violation of security policies (Shin et al., 2010). CWE and CVE are two of the most important standards used to identify vulnerabilities. CWE defines vulnerability types (not specific instances) and CVE defines specific instances within a product or a system.

A typical example of a CWE can be CWE-476 - NULL Pointer Dereference. Its instance in TensorFlow is CVE-2021-37638 (in file tensorflow/core/kernels/ragged_tensor_to_tensor_op.cc). A missing check results in a NULL pointer dereference when “row_partition_types_” is empty and the code tries to access its first element. Below, we present a fix of this vulnerability (a yellow line was the one allowing vulnerability exploitation and the other ones had to be added to fix it - a missing if statement and return from the function with the corresponding error) (Commit fixing CVE-2021-37638 vulnerability, 2021):

```
1 if (row_partition_types_.empty()) {
2     return errors::InvalidArgument("No row_partition_types given.
3 ");
4 }
5 const RowPartitionType first_partition_type = row_partition_types[0];
```

Listing. 1: Example of the CWE-476 vulnerability fix in the TensorFlow implementation

2.2. Vulnerabilities regarding artificial intelligence

Due to complexity, large scope and constant development of the AI domain, the AI threat landscape is still not completely defined and difficult to map. It is also impossible to predict all of the future scenarios, especially when DL algorithms are applied to solve problems that are troublesome for humans. Some of the threats dedicated to AI include compromising AI inference, model poisoning and sabotage, data poisoning (malicious actors inject false training data to corrupt the learned model (Steinhardt et al., 2017)) and tempering, as well as manipulation of data labelling. Models can also be tricked using adversarial examples (Responsible AI practices Google AI, 2021).

Abstracting from these dedicated threats, which we cannot fully combat, there are other significant threats that are often overlooked in the AI threat research papers - compromises of model frameworks and 3rd party provider failures. Many companies use external (often open-source) machine learning libraries to power their solutions. This 3rd party software itself can (and probably does) contain many software vulnerabilities, especially those using C/C++ for speed and high resource control. They are no less significant than threats crafted specifically against AI. Moreover, these are the threats that we know and are able to handle and implement some mitigation strategies. That is why, in this study we focus on these particular vulnerabilities.

2.3. Vulnerability detection

The approaches to software vulnerability detection can be divided into two main groups: Static Analysis and Dynamic Analysis (Ghaffarian and Shahriari, 2017). Dynamic analysis requires an executable version of a product and is usually very time-consuming, as it needs to test a sufficient number of test cases (Zhioua et al., 2014). Static analysis can be applied directly to source code, thus we use it to examine the TensorFlow implementation. Programs checking the source code files (based on a set of predefined rules (Pereira et al., 2021b)) are called Static Analysis Tools (SATs). A compilation of static analysis products with a description of supported languages, availability and a brief characterization can be found on the OWASP organization's website (Source Code Analysis Tools - OWASP). Alternatively, one can use a list provided by The National Institute of Standards and Technology (NIST) (Source Code Security Analyzers - NIST). The examples of such programs dedicated for C/C++ are CppCheck (CppCheck, 2021), FlawFinder (FlawFinder, 2021) and Visual Code Grepper (Visual Code Grepper, 2022). These tools are commonly used in the literature Croft et al. (2021); Kaur and Nayyar (2020); Khaled and Abdelbaki (2020); Li et al. (2018); Pereira et al. (2021a); Riom et al. (2021); Zhao et al. (2021) to examine general-purpose products. In this study, we use them to examine their ability to find known and potentially unknown issues in TensorFlow.

2.4. Related work

ML-based Vulnerability Detection Some recent works use ML methods to find vulnerabilities. In Medeiros et al. (2020), different ML-based vulnerability detection methods have been tested. In Filus et al. (2021), metrics generated via SonarQube and CCCC tool were used (with ML models) to detect resource management error vulnerabilities (CWE-399) and buffer error vulnerabilities (CWE-119). The study showed that the detection effectiveness depends on the CWE type. In Filus et al. (2020), the system based on neural networks and mixed features (text and from SonarQube) was tested to detect CWE-399 vulnerabilities (Filus et al., 2020). In Li et al. (2018), VulDeePecker was proposed - a vulnerability detection DL system - and tested on CWE-119 and CWE-399 vulnerabilities.

Using SATs for vulnerability detection Although the ML models are widely used in the area of vulnerability detection, the traditional rule-based Static Analysis Tools are still commonly used. In Pereira et al. (2021b), Buffer Overflow Vulnerabilities (CWE-119) have been analyzed using ODC and two SATs. In Arusoae et al. (2017), eleven C/C++ SATs were benchmarked based on the Toyota test suite (Shiraishi et al., 2015) (it was also used in Nong et al. (2021), in which different tools were tested to find memory-related vulnerabilities). Also works Croft et al. (2021); Kaur and Nayyar (2020); Khaled and Abdelbaki (2020); Riom et al. (2021); Zhao et al. (2021) aim to benchmark different SATs. Their findings showed us the potential of three SATs in the context of vulnerability detection: CppCheck, FlawFinder and Visual Code Grepper. We use these findings as a guideline and test the effectiveness of these SATs in terms of finding known vulnerabilities in the TensorFlow implementation. Nevertheless, software developers can also reach for other alternative products. A compilation of static analysis products with a description of supported languages, availability and a brief characterization can be found on the OWASP organization's website along with the important selection criteria (Source Code Analysis Tools - OWASP). Alternatively, one can use a list provided by The National Institute of Standards and Technology (NIST) (Source Code Security Analyzers - NIST). It is also worth mentioning a different kind of a static analysis tool, which can be used along with the described

products - code clone detectors. Such tools aim to find fragments of code copied and pasted within/between software products. The example of such a product is VUDDY (Kim et al., 2017), which identifies code clones of known CVEs (exact and renamed clones in particular). Such tools can be effective in finding copied parts of vulnerable code, however, they show low effectiveness in finding restructured and semantic clones of such vulnerabilities (Kim et al., 2017). In fact, restructured and semantic clones are excluded from the VUDDY clone detection (Kim et al., 2017). On contrary, the findings presented in our work, do not focus on the code clones, but aim to summarize the semantic meaning of code, near which vulnerabilities are often introduced. As a result, our main findings provide a more general description of potential security hot-spots. Nevertheless, tools such as VUDDY can be helpful, when programmers reuse some parts of code from the open-source libraries.

Vulnerabilities in AI products Although numerous works regard software vulnerability detection based on Deep Learning models (Fang et al., 2019; Filus et al., 2020; Li et al., 2021; 2018; Pechenkin and Demidov, 2018), works on vulnerabilities in AI products themselves are very limited. One work that we were able to find, Xiao et al. (2018), discusses security risks in DL frameworks (TensorFlow, Caffe and Torch). However, they examine vulnerabilities mainly in their dependencies (i.e. NumPy for TensorFlow, opencv for Caffe/Torch) - and not directly in DL frameworks code. Given the ubiquity of DL models in a variety of applications, this is a large gap in the literature.

Limitations of the related works Besides the lack of a thorough analysis of vulnerabilities in ML products, another limitation of the existing works is that they analyze vulnerabilities regardless of their CWE type or a very limited number of vulnerability types. Moreover, they do not focus on their significance and severity.

Our work in relation to the available literature With respect to the works present in the literature, our work can be embedded closest to that of Pereira et al. (2021b) in terms of methodology and Xiao et al. (2018) in terms of subject matter. Our work aims to fill the gap between them. First of all, we aim to focus on the implementation of a deep learning framework itself, and not on its dependencies, as in work (Pereira et al., 2021b). In terms of methodology, work (Xiao et al., 2018) provides a systematic analysis of CWE-119 issues (Improper Restriction of Operations within the Bounds of a Memory Buffer). Authors examined three C/C++ applications: Linux Kernel, Xen and Mozilla. In contrast to these old products, we examine a relatively new product in which at least some changed programming practices can be expected (although the development of Linux Kernel, Xen and Mozilla is continuous, they have never been rewritten from scratch). Moreover, the numerical libraries, such as TensorFlow, mostly rely on mathematical operations using arrays. Here, the root causes of vulnerabilities can be different - e.g. they can result from an insufficient knowledge of the mathematical methods to be implemented (assumptions, boundary conditions, etc.). We aim to identify specific root causes of different CWE types. We take into consideration three types of memory issues: not only a generic memory-related type - CWE-119, but also its "children": Out-of-bounds Read and Write. We also analyze problems related to NULL Pointer Dereference (CWE-476) and Integer Overflow or Wraparound (CWE-190) - other common C/C++ issues that are critical for numerical operations and dealing with dynamic data structures. We also analyze issues related to Improper Input Validation (CWE-20) - as one example of a language-independent CWE group. Work (Pereira et al., 2021b) shows that SATs lack rules to detect most examined vulnerabilities, however they can effectively find some memory-related vulnerabilities in traditional products. Therefore, we aim to examine whether free SATs can be used to improve the security of a different product kind - a modern numerical library.

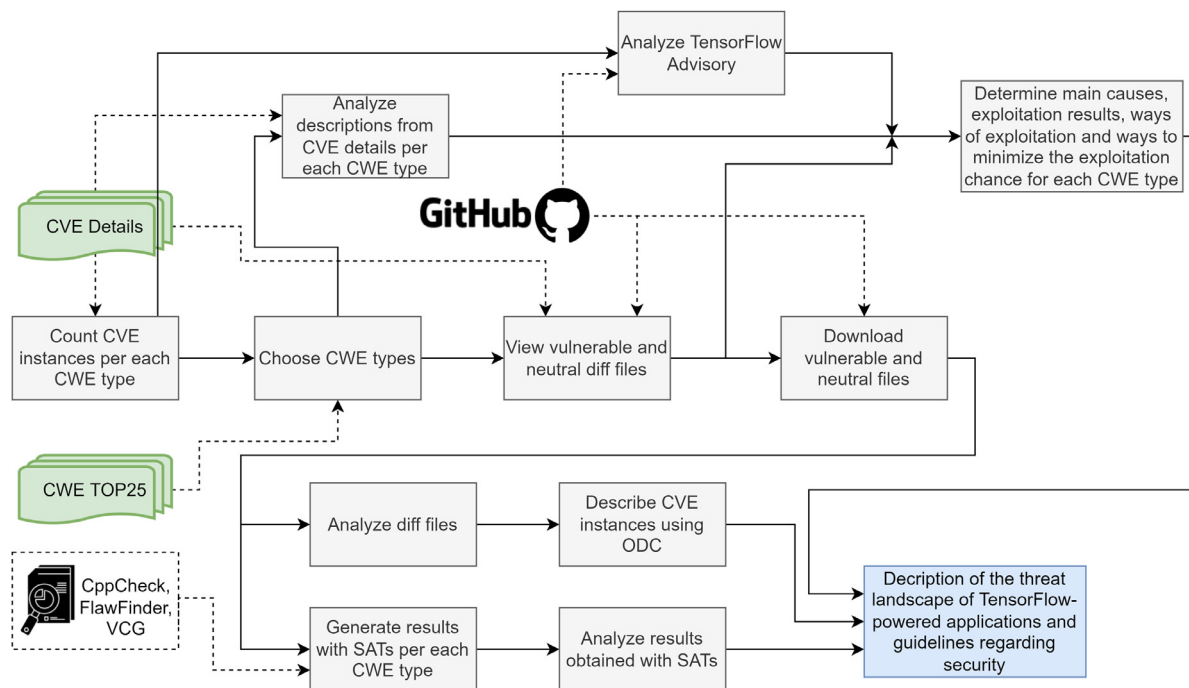


Fig. 1. The overview of the approach used to analyze the TensorFlow vulnerabilities.

3. Approach to analysis

In our study, we aim to analyze different types of software vulnerabilities in terms of their severity, impact on confidentiality, integrity and availability, as well as possible results of exploitation. We also analyze the characteristics of different vulnerability types and determine specific situations, in which these particular types can be introduced. We test three static code analysis tools to examine their effectiveness in finding known software vulnerabilities.

We use three sources of data: CVE Details, CWE TOP25 ranking and GitHub. We use CVE Details to find known instances of vulnerabilities in TensorFlow. CVE instances are connected with a specific CWE type. We use this knowledge along with information from CWE TOP25 to find the most prevalent and severe vulnerabilities. This way we consider TensorFlow as an individual product (a numerical library) as well as an example of a modern C/C++ application. Then, we use CVE Details for the second time - to find CVE instances under a specific CWE group and gather metadata regarding those vulnerabilities. We take into consideration only those CVE pages that provide a single GitHub commit fixing a specific vulnerability. We download vulnerable (before a fix) and neutral (after a fix) files. We inspect the files manually (with ODC) and via static code analyzers. As a results, we provide some numerical results regarding the effectiveness of SATs and statistics regarding the examined vulnerabilities' metadata. We also provide a set of insights regarding different types of vulnerabilities and a list, in which we include specific situations, in which programmers can introduce security issues. We present this list in a form of a checklist for programmers of numerical libraries, programmers of applications using such libraries, as well as creators of SATs (they can use it to create new rules or to determine test cases for their solutions). In [Fig. 1](#) we present the overview of the steps of our analysis.

3.1. Selection of CWE types and CVE instances

To choose the CWE types we considered the number of available TensorFlow CVE instances per CWE type and its position in

the 2021 CWE TOP25 ranking ([CWE Top 25 Most Dangerous Software Weaknesses, 2021](#)). This way, we take into consideration three different aspects of different vulnerability types: their prevalence and severity in software products in general, as well as their prevalence in TensorFlow. CVE Details provides us with information regarding the occurrence of known vulnerability types in TensorFlow. CWE TOP25, on the other hand, takes into consideration not only the prevalence of different vulnerability types in diverse applications, but also the severity of these vulnerabilities and their impact on the system in the case of exploitation. This allows us to treat TensorFlow individually and also as an example of a modern C/C++ application.

Eventually, we included 6 CWE types (**CWE - Common Weakness Enumeration, 2021**) (104 CVE instances in total). In [Table 1](#), we gathered all the examined CWE types along with the number of their instances in TensorFlow and their rank in the CWE TOP25 ranking. The examined vulnerabilities seem to be adequate from a security point of view for a numerical library. Such libraries often rely on mathematical operations using arrays. Although languages such as C or C++ offer speed calculations, they do not provide built-in protection against memory access/overwriting or checks regarding buffer bounds. Our analysis includes a more generic memory-related CWE type - CWE-119, but also its “children”: CWE-125 and CWE-787 as more specific types of memory issues. We also analyze different problems characteristic for numerical operations and dealing with dynamic data structures: related to NULL Pointer Dereference (CWE-476) and Integer Overflow or Wraparound (CWE-190). We also analyze issues related to Improper Input Validation (CWE-20) - as one example of a language-independent CWE group affecting all kinds of applications.

3.2. Retrieval of metadata for selected CVE instances and collection of source code files

We used CVE Details page to find the CVE metadata and links to github commits with the source code. On the particular CVE page, information on CVSS Score, impact on CIA triad (three levels are used: None, Partial and Complete) and potential exploitation

Table 1
CWE types used in the study. We sorted the types according to the descending number of instances. Rank denotes a rank in the CWE TOP25 ranking.

Instances	Rank	CWE	Name	Description
34	#3	CWE-125	Out-of-bounds Read	As a result, data can be read after the end or before the beginning of the allocated buffer.
25	#15	CWE-476	NULL Pointer Dereference	It occurs when a given program dereferences a pointer that is expected to be valid, but is in fact NULL.
19	#1	CWE-787	Out-of-bounds Write	As a result, data can be written after the end or before the beginning of the allocated buffer.
11	#4	CWE-20	Improper Input Validation	It is caused by no or incorrect validation of the input.
8	#17	CWE-119	Improper Restriction of Operations within the Bounds of a Memory Buffer	Operations on a memory buffer cause a read from or a write to a memory location that is outside of the intended buffer boundaries (parent of CWE-787 and CWE-125).
7	#12	CWE-190	Integer Overflow or Wraparound	Calculation of a value can produce an integer overflow/wraparound, while the result should always be larger than the original value.
Total: 104				

results can be found. We gathered the necessary data and determined the common statistics describing different CWE types. To manually analyze the code, we used GitHub commits fixing the CVE instances. We used the GitHub viewer for ODC and we downloaded the vulnerable and neutral files for the analysis with SATs (vulnerable files are versions before the fix, whereas the neutral files include the fix).

3.3. Orthogonal Defect Classification

Orthogonal Defect Classification (Chillarege et al., 1992) is a systematic approach developed by IBM, used to classify and analyze software defects by transforming semantic information into a measurement (Pereira et al., 2021b). The approach is frequently used in software root-cause analysis (Pereira et al., 2021b). Full analysis involves describing the defect before and after a fix with a series of 8 attributes (with predefined values). The following attributes can be used: activity, trigger, impact (attributes regarding an opened defect), target, defect type, qualifier, source and age (attributes regarding an opened defect). To examine vulnerabilities in our study, we use two attributes: **defect type** and **qualifier** (the others do not include any additional relevant information regarding the characteristics of a particular vulnerability (Pereira et al., 2021b)).

Attribute **defect type** is used to represent the semantics of the fix (it captures the meaning of the fix, thus the nature of the particular issue)(Chillarege et al., 1992). Eight values can be assigned to this attribute. We exclude two of them from our analysis (build/package/merge and documentation, as we focus only on the source code files). Based on definitions from Chillarege et al. (1992) and Bridge and Miller (1998), we can define the defect types in the following way:

- **Assignment/Initialization** - problems related to no/incorrect assignment of the variable value.
- **Checking** - problems related to conditional logic (incorrect, missing conditions). When multiple checking statements have to be added/corrected it can belong to type Algorithm/Method.
- **Timing** - problems related to serialization of shared resources.
- **Algorithm/method** - problems related to implementation (generally no design fix should be necessary).
- **Function** - problems related to no or incorrect implementation of some significant capability, end-user interfaces or global data structures (generally, it does require a design change).
- **Interface** - problems related to communication issues between subsystems, components, device drivers etc.

Types above have been described in more detail in work Bridge and Miller (1998).

Qualifier is used to capture what had to be done to fix a particular vulnerability in terms of code changes:

- **Missing** - to fix the issue, new code had to be added.
- **Incorrect** - to fix the issue, some code had to be adjusted.
- **Extraneous** - to fix the issue, some code had to be removed.

We can say, that it extends the semantic information provided via a defect type. It provides us with insights on whether a particular defect resulted from an omission or rather from some kind of a programming mistake.

Using these two attributes, allows us to associate a vulnerability with particular activities of different stages of software development and faults in this process (Chillarege et al., 1992). They provide us with the causal information regarding an analyzed vulnerability. The resulting insights describe the nature of different vulnerability types in a systematic and easy to interpret way.

In our analysis, we select appropriate attribute values based on the changes occurring during the patch of a given vulnerability. The changes can be easily inspected via github - by displaying the corresponding commit.

The ODC is a manual classification, so similarly to work (Pereira et al., 2021b), two researchers conducted the analysis for greater independence and fault tolerance. First, we performed the discussion, set the classification criteria and analyzed the first several examples together. We separately analyzed all of the remaining CVE instances, and at the end, discussed the non-overlapping results and found a common consensus - this is how the final results have been obtained.

3.4. Running SATs and analysis of the alerts

We used three commonly used static code analyzers: CppCheck (version 2.6), FlawFinder (version 2.0.19) and Visual Code Grepper (version 2.2.0.0). We decided to use only SATs that can be used for free - as those SATs can be used by everyone. We chose these particular SATs as they showed superior performance over the other analyzers tested in the works (Croft et al., 2021; Kaur and Nayyar, 2020; Khaled and Abdelbaki, 2020; Riom et al., 2021; Zhao et al., 2021). In work Riom et al. (2021), 9 SATs were tested. Cppcheck was reported to produce very few false positive alerts and specialize in finding undefined behaviours. Also in work Kaur and Nayyar (2020), it was shown that CppCheck was able to find the maximum number of the examined vulnerabilities out of the tested

tools. In work [Khaled and Abdelbaki \(2020\)](#), the results showed that although the Visual Code Grepper SAT gives a relatively large number of false positive alerts, it was able to report all the examined vulnerabilities (false negative rate equaled zero). In work [Zhao et al. \(2021\)](#), Flawfinder significantly outperformed another tested traditional SAT in terms of finding CWE-119 memory-related issues and F1 Score. Also in work [Croft et al. \(2021\)](#), Flawfinder was able to produce the highest recall values out of all tested SATs.

We first ran the analysis on vulnerable files corresponding to the chosen CWE group, and then we analyzed the respective neutral files. We did this for all the examined CWE groups. Different levels of granularity can be used in the process of software vulnerability detection: among others package-level, file-level, function-level and line-level ([Kluban et al., 2022](#)). Usually, at least a function-level is recommended, as it provides sufficient information regarding a code functionality - unlike separate lines of code ([Kluban et al., 2022](#)). While the Machine Learning models built to detect vulnerabilities usually operate on a function-level (e.g. works [Russell et al. \(2018\)](#); [Wei et al. \(2021\)](#)), the file-level seems to be the most suitable in our case, as we aim to perform the overall assessment of adequacy of different warnings per file. We take into consideration not only a particular vulnerability, but also the warnings regarding the qualitative improvements, which can reduce the chance of introducing new vulnerabilities in the future. We believe that such an evaluation will be the most useful for those who would like to use these static code analyzers to find security issues and to improve the overall quality of their code.

When the files were analyzed for a specific CWE group, we analyzed the alerts in terms of their CWE ID (when given), accuracy of finding lines regarding the examined CVE instances (we used the information regarding the lines for which SATs found issues, found them in files regarding specific vulnerabilities on GitHub and compared with true lines that were responsible for some vulnerability). We also analyzed whether the given alerts should be considered (we chose alerts that can be used to improve the quality of TensorFlow code and described the potential issues and improvements). We also compared the results obtained for corresponding vulnerable and neutral versions of files. In the case of VCG, which offers an analysis of comments (to find potentially unfinished fragments of code), we chose two example files and examined how many "TODO" instances are found by VCG and how many instances of such comments truly are in a particular file (we used a Visual Studio Code search engine for this purpose and simply typed "TODO"). We computed the number of false positives (false alarms - when a comment is just an explanation of the code) and false negatives (when a clear "TODO" can be found and it was not reported by VCG).

3.5. Analysis of text descriptions of vulnerabilities and source code elements

After our primary analysis of information on CVE Details and ODC, we focused on the vulnerability descriptions regarding all of the examined CVE instances available on CVE Details and corresponding TensorFlow advisories, as well as code implementations (if necessary) to find common causes and results of vulnerabilities occurrence and exploitation. We defined possible exploitation strategies of the users of application using TensorFlow background and possible ways to avoid their exploitation in a form of a checklist

4. Analysis of information from CVE Details

Using information gathered from CVE Details, we have examined metadata regarding the examined CVEs: Common Vulnerability Scoring System (CVSS) scores and the impact on Confidential-

Table 2

Possible results of exploits of different vulnerability types.

	Bypass	DoS	Code Exec.	Mem. Corr.	Overflow
CWE-787	21.05%	0.00%	5.26%	5.26%	68.42%
CWE-20	0.00%	72.73%	0.00%	0.00%	0.00%
CWE-119	0.00%	12.50%	12.50%	0.00%	100.00%
CWE-125	2.94%	2.94%	0.00%	0.00%	8.82%
CWE-190	0.00%	42.86%	0.00%	0.00%	100.00%
CWE-476	4.00%	12.00%	8.00%	0.00%	0.00%

ity, Integrity and Availability (CIA triad) of a system when a vulnerability is exploited. Common Vulnerability Scoring System measures the severity and characteristics of software vulnerabilities. A score ranges from 0 to 10. CVSS is considered to be a standard measurement system for organizations, governments and in industry, as it provides consistent and accurate vulnerability severity scores (according to National Institute of Standards and Technology - NIST ([National Vulnerability Database \(NVD\), 2021](#))). CIA triad, on the other hand, refers to the fundamental elements of information systems: Confidentiality, Integrity and Availability ([Samonas and Coss, 2014](#)). Three categories are used to describe the impact on each CIA triad part: none, partial and complete ([Toloudis et al., 2016](#)).

To gather our metadata, we used the CVE details searching engine ("Vulnerability Search") and typed TensorFlow in the "Product" box. We exported all available data to a csv file and processed it. We calculated average, standard deviation, minimum and maximum values of the CVSS scores for different types of examined vulnerabilities. We also counted the CVE instances from each CWE group belonging to different categories of the CIA triad impact (for all parts of CIA triad separately). We used these values to compute the percentage of these impact categories per each CWE type. The results can be observed in [Fig. 2](#) and [Tables 2](#) and [3](#). CWE groups in [Fig. 2](#) have been sorted in the descending order of the average CVSS score.

It can be observed that the highest values of CVSS score have been obtained for memory-related issues (CWE-787, CWE-119 and CWE-125) - 7.5 (they usually impact all of the CIA triad elements). These vulnerabilities are also the most diversified in terms of possible exploit results (overflows, bypasses, code executions, memory corruptions and DoS - see [Table 2](#)). CWE-190 group (Integer Overflow or Wraparound) obtained the lowest CVSS values on average (they usually impact only Availability - see [Table 3](#)). CWE-190 vulnerabilities result in overflows and DoS. The examined CWE-20 examples (Improper Input Validation) obtained higher average and maximum values than CWE-190 vulnerabilities (also, the highest standard deviation - suggesting the most varying CVSS scores). These vulnerabilities have an impact mainly on Availability of systems (via Denial of Service - see [Table 2](#)). The results obtained for CWE-476 vulnerabilities are similar to those obtained for CWE-20 in terms of statistics of CVSS scores (almost the same mean, std and the same minimum and maximum values). Also these vulnerabilities impact mainly Availability, but in contrast to CWE-20 vulnerabilities, almost 50 percent of them can also impact Confidentiality or Integrity. For the majority of the examined CVE instances, the exploitation results were not classified to any of the given groups, but some of the vulnerability instances can result in bypasses, denials of service and code execution.

5. Orthogonal Defect Classification

The results of ODC are to some extent similar for all the CWE types: in no case "Qualifier" attribute took the value "Extraneous" and "Defect Type" attribute - "Timing" and "Assignment" (therefore

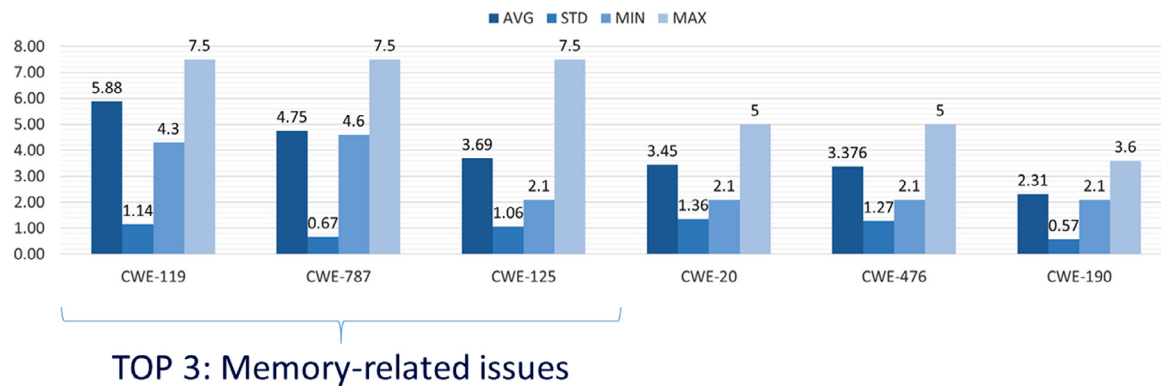


Fig. 2. Average, standard deviation, minimum and maximum values of the CVSS score for different types of examined vulnerabilities. CWE groups have been sorted in the descending order of the average CVSS score.

Table 3

Impact of different vulnerability types on CIA triad.

	Confidentiality			Integrity			Availability		
	None	Partial	Compl.	None	Partial	Compl.	None	Partial	Compl.
CWE-787	0.00%	100.00%	0.00%	0.00%	100.00%	0.00%	0.00%	100.00%	0.00%
CWE-20	90.91%	9.09%	0.00%	90.91%	9.09%	0.00%	0.00%	100.00%	0.00%
CWE-119	25.00%	75.00%	0.00%	25.00%	75.00%	0.00%	25.00%	75.00%	0.00%
CWE-125	11.76%	88.24%	0.00%	82.35%	17.65%	0.00%	14.71%	85.29%	0.00%
CWE-190	100.00%	0.00%	0.00%	85.71%	14.29%	0.00%	0.00%	100.00%	0.00%
CWE-476	56.00%	44.00%	0.00%	52.00%	48.00%	0.00%	0.00%	100.00%	0.00%

Table 4

Results of ODC obtained for different types of vulnerabilities.

	CWE-787			CWE-20		
	Missing	Incorrect	Total	Missing	Incorrect	Total
Checking	53%	16%	68%	45%	9%	55%
Algorithm/Method	5%	26%	32%	18%	9%	27%
Function	0%	0%	0%	0%	9%	9%
Interface	0%	0%	0%	0%	9%	9%
Total	58%	42%		64%	36%	
	CWE-119			CWE-125		
	Missing	Incorrect	Total	Missing	Incorrect	Total
Checking	38%	13%	50%	68%	9%	76%
Algorithm/Method	38%	13%	50%	6%	9%	15%
Function	0%	0%	0%	0%	6%	6%
Interface	0%	0%	0%	0%	3%	3%
Total	75%	25%		74%	26%	
	CWE-190			CWE-476		
	Missing	Incorrect	Total	Missing	Incorrect	Total
Checking	14%	14%	29%	56%	24%	80%
Algorithm/Method	14%	57%	71%	4%	8%	12%
Function	0%	0%	0%	0%	0%	0%
Interface	0%	0%	0%	0%	8%	8%
Total	29%	71%		60%	40%	

we do not include them in our results). The detailed results of ODC for all the CWE types can be found in Table 4.

For the memory-related vulnerability types (CWE-787, CWE-125 and CWE-119), the most common problem was missing checking statements. For CWE-119, the same results were obtained for “Checking” and “Algorithm/Method”, indicating the need for more extensive improvements. This type of CWE is the parent type of the other memory-related CWE-types, so it can deal with more diverse cases. The prevalence of “Missing” values for the attribute “Qualifier” means that probably in the implementation phase of some functionalities, the need for simpler or more complex validation of some parameters was forgotten/not considered. The same problem can be observed for CWE-20. For CWE-125 also values “Function” and “Interface” of attribute “Defect Type” were observed, which suggests a more complex nature of the defect causing the vulnerability as it requires a design and communication change.

The only CWE type for which more defects requiring code adjustments rather than just additions were observed was CWE-

190 (Integer Overflow or Wraparound). Moreover, most of the defects required rebuilding a larger part of a given code fragment to be fixed (“Defect Type”: “Algorithm”). For CWE-476, the ODC showed that the problem was mainly missing checks (which is logical, since in many cases to fix a vulnerability, a check is needed whether the pointer to be accessed is not a nullpointer).

6. Analysis of SAT alerts

We used three open-source static code analyzers (CppCheck, FlawFinder and Visual Code Grepper) for vulnerable and neutral versions of the source code files for all the CWE groups. In Table 5 we summarised the number of issues reported for different CWE types and SATs and reflected how this number differs in code versions before and after the fixes. For the majority of CWEs, it was the Visual Code Grepper that reported the largest number of issues. We focused especially on the cases, in which the number of issues changed between vulnerable and neutral versions. The smallest number of issues was reported by FlawFinder. We have man-

Table 5

Number of issues found by FlawFinder, CppCheck and Visual Code Grepper Static Code Analyzers.

	FlawFinder		CppCheck		VCG	
	Vuln.	Neutral	Vuln.	Neutral	Vuln.	Neutral
CWE-787	7	7	7	7	16	16
CWE-20	9	9	16	16	27	27
CWE-190	1	1	2	2	15	16 ↑
CWE-119	0	0	10	10	0	0
CWE-125	23	21 ↓	27	27	61	60 ↓
CWE-476	48	48	35	35	79	79

ually checked all the reported issues. In Sections 6.1, 6.2 and 6.3, we provide the discussion regarding the adequacy of the reported warnings. We also provide a discussion and a few examples of false positive and false negative alerts on the example of VCG for clarity. Apart from vulnerability detection, we present some example SAT suggestions we found useful - mainly qualitative, stylistic fixes that affect the clarity of code. We believe, that introducing these qualitative changes could prevent introduction of new vulnerabilities in the future.

We can summarize our findings as follow: CppCheck performs a more in-depth analysis in addition to simple checks for "unsafe" functions. Visual Code Grepper besides code, also analyses comments - to find phrases that indicate broken or unfinished code (regarding e.g. TODOs or fixes). FlawFinder performs simple checks for "unsafe" functions - its practical application to improve security is negligible (the same functionality is realized by the other SATs in addition to other functionalities).

6.1. Flawfinder

For the majority of CWE groups, the number of issues was exactly the same for vulnerable and neutral versions of files: **CWE-787**: 7 issues found, **CWE-20**: 9 with 1 CWE-20/CWE-807, **CWE-190**: 1, **CWE-119**: 0, **CWE-476**: 48. The only exception was CWE-125 group, where FlawFinder found 23 issues for vulnerable and 21 for neutral versions of the same files. The difference was caused by the fact that the fix of CVE-2020-26267 removed unnecessary calls to 'strings::StrCat' [Commit fixing CVE-2020-26267 vulnerability \(2021\)](#).

Issues with the following CWE IDs were found: **CWE-120** (Buffer Copy without Checking Size of Input), **CWE-327** (Use of a Broken or Risky Cryptographic Algorithm), **CWE-20** (Improper Input Validation) and **CWE-807** (Reliance on Untrusted Inputs in a Security Decision). Generally, they turned out to be false alarms - instances that can suggest the occurrence of a vulnerability, but in a particular context the warning is not appropriate. E.g. CWE-327 warnings were reported for *random* function usages - these are false alarms, since *random* was not used for security purposes in the analyzed context. Problems related to CWE-120, considered the use of *memcpy* and *StrCat* functions. **The great majority of StrCat instances could be removed**, as function "InvalidArgument" from "tensorflow/tensorflow/core/platform/errors.h" concatenates strings given as arguments - no StrCat is required. Majority of *memcpy*-related warnings seem to be false alarms and are placed nowhere close to actual vulnerabilities. Nevertheless, one *memcpy* issue was reported by the SAT ("memcpy: Does not check for buffer overflows when copying to destination (CWE-120). Make sure destination can always hold the source data.") that was included in the fix of CVE-2021-37637 vulnerability (CWE-476) in commit ([Commit fixing CVE-2021-37637 vulnerability, 2021](#)) (along with other necessary changes, as this vulnerability refers to more parts of code). Below we present the fix, yellow lines are those

causing the vulnerability, the remaining ones are the fix (missing if condition):

```
1 if (buffer) {
2     memcpy(position, buffer->data(), buffer->size());
3     metadata->set_tensor_size_bytes(buffer->size());
4 }
```

One potential CWE-20 vulnerability was detected for the CWE-20 group ("Environment variables are untrustable input if they can be set by an attacker. ... Check environment variables carefully before using them."), however it was placed in a completely different place than the examined one in file "tensorflow/stream_executor/cuda/cuda_dnn.cc" - and it refers to a helper class for cudnn-related algorithms ([Commit fixing CVE-2020-26270 vulnerability, 2021](#)).

6.2. CppCheck

For all of the CWE groups, the results were exactly the same for vulnerable and neutral versions of files (see Table 5). **In no case the instance of a true CWE type was found by CppCheck.** Issues with the following CWE IDs were found: **CWE-398** (Code Quality), **CWE-563** (Assignment to Variable without Use), **CWE-571** (Expression is Always True), **CWE-197** (Numeric Truncation Error). Generally, the great majority of warnings were of type CWE-398 (Quality) and considered e.g. raw loops, local variable shadowing outer argument, reducing the scopes of some variables and the presence of the same if conditions - some of them were reasonable and changes could be applied. This prompted us to include one of the suggestions that can be used to improve the code quality. For files ([Commit fixing CVE-2021-29566 vulnerability \(2021\)](#)) related to CVE-2021-29566 (CWE-787), CppCheck returned the following warning: "Local variable 'd' shadows outer argument (CWE-398)":

```
1 // TODO(gpapan): Write multi-threaded implementation
2 ...
3 void operator()(const CPUDevice& d, typename TTypes<T, 4>::
4     ConstTensor input,
5     for (int d = 0; d < depth; ++d)
```

It is not a good practise to redefine a function argument and use it as a counter in a loop - it should be fixed. A different variable should be used here to obtain better code quality. A comment shows that it is planned to write a multi-thread implementation of this function and potentially *const CPUDevice& d* will be used there (which is now shadowed). Similar issues were found, e.g. variable "i" in file tensorflow/core/kernels/save_restore_v2_ops.cc. It can often be tempting to use variables such as "i", "d" or other single letters as loop counters, but it should always be checked whether this variable is not already taken - this, even if it does not cause problems at the moment, may introduce problems in the future (e.g. when the code is rebuilt by another programmer).

6.3. Visual Code Grepper

Unfortunately, Visual Code Grepper does not provide Common Weakness Enumeration IDs in its reports. It only provides warnings with different tags (e.g. "potentially unsafe", "suspicious comment", "critical"). The highest severity of a warning was "high" - no cases with "critical" warnings were observed. Besides analyzing code, VCG also processes comments and analyzes its contents via text analysis (e.g. it searches for terms such as "TODO"). In some cases it can be useful - e.g. see Section 6.2 for an example of a planned functionality. It also finds StrCat and memcpy instances and suggests replacing them with different functions, as those functions appear in Microsoft's banned function list. VCG returns warnings when it finds multiple deletes for a particular variable or when it cannot find a delete for a new (in the cases examined by us manually, these were false positives - they regarded the

situations, in which a pointer was created inside a function and then returned). VCG also finds instances of using “getenv” - these should be handled with caution. VCG returns warnings regarding Signed/Unsigned Comparison” as a beta functionality: “This behaviour can return unexpected results as negative numbers will be forcibly cast to large positive numbers”. In the examined cases, all of the warnings seem to be false positives and not actual security hot-spots: signed variables in question are created inside a for loop and only incremented (no negative values occur). Also, for file tensorflow/compiler/mlir/lite/transforms/optimize.cc (CWE-476), VCG returned the following warning: “The ‘assert’ macro usually only exists for code in the debug build.” Basically, no check will take place in the production code for line 279. In this place it seems to be responsible for some kind of error handling:

```
1 assert(elements.getNumElements() == shape.back());
```

so it could be changed.

A different value of warnings can be observed for the CWE-125 group in Table 5. The same as for FlawFinder, VCG shows that one StrCat instance has been removed within a vulnerability fix (when errors::InvalidArgument is used - see Section 6.1). For CWE-190, the number of issues is larger for the neutral case, because the fix of the CVE-2021-29605 vulnerability introduced a warning regarding malloc without free. It is a false alarm - memory is allocated for the variable to be returned by the examined function.

What is the accuracy of the VCG text analysis performed for comments? We manually analyzed the reported alerts and the corresponding source code files for two example files (before vulnerability fixes), for which VCG returned a large number of warnings regarding comments. We counted these warning. Then, we used Visual Studio Code to search for phrase “TODO” (the most obvious sign of the unfinished code or the code to be corrected) - which should theoretically be found with VCG too. We analyze two files: “tensorflow/stream_executor/cuda/cuda_dnn.cc” (CWE-20) and “tensorflow/lite/kernels/internal/optimized/optimized_ops.h” (CWE-476). Generally, when the parenthesis is used just after the “TODO”, the “TODO” is not detected. Such instances can be perceived as false negatives - these are potential parts of code that should be finished or fixed. In the case of cuda_dnn.cc it was 12 instances, and for optimized_ops.cc - 28 instances. On the contrary, VCG searches for terms such as “to do”, which often leads to reporting false positives (when to do = to perform). False positives regarding comments denote comments that seem to explain the code parts and are not alarming to fix/add something. In 2 cases for cuda_dnn.cc, and in 6 cases for optimized_ops.cc, VCG reported a warning when the comment seems to be just an explanation (we treat them as false positives). Naturally, not all TODOs imply security issues, but should be reviewed by the programmers, because potential security hotspots can be found here (unfinished/incorrect code). Below we provide some examples of parts we perceive False positives and False negatives from the file “tensorflow/stream_executor/cuda/cuda_dnn.cc” file before the CVE-2020-26270 vulnerability fix:

- Example of a false positive:

```
1 // The reason that we only support the kBatchYXDepth output
   layout
2 // is that we have to do
3 // something in the depth for each (y,x)
4 // coordinate.
```

- Examples of false negatives:

```
1 LOG(FATAL) << "not yet implemented"; // TODO(leary)
```

or

```
1 // TODO(yangzihao):
2 //Test with negative dilation to make sure that cudnn
3 // doesn't crash.
```

It seems that VCG is unable to find TODO statements when they are used with parentheses with no spacing (just after the word TODO). Our own search for TODO in the examined file showed that they include many fragments yet to be finished.

7. Analysis of descriptions, advisories and source code

Our manual analysis resulted in the identification of common effects of exploitation of different CWE types:

- **CWE-787**: heap buffer overflows (writes and reads), segfaults, corruptions of the memory. Chained with other types can result in more advanced exploits and can trigger undefined behavior by null pointer dereference.
- **CWE-20**: access to data outside the bounds of heap allocated data, segmentation faults, assertion failures, DoS (by query-of-death vulnerability, ‘SIGABRT’ signal raised by the operating system or ‘CHECK’-failure), data corruption and undefined behaviors.
- **CWE-190**: dereferencing invalid pointers - which can cause memory allocation problems and denials of service (via ‘CHECK’-fail).
- **CWE-119**: read from outside the bounds of the heap buffer allocated buffers, segmentation faults, data corruption (by accessing data out of bounds), a heap buffer overflow (by accessing data out of bounds), undefined behaviors and DoS (via out of memory allocation).
- **CWE-125**: reads/writes outside of bounds of heap allocated data (and disclosure of the contents of process memory), segmentation faults, undefined behaviors due to binding to nullptr, uninitialized memory accesses and DoS.
- **CWE-476**: segmentation faults, read from/write to memory outside the bounds of heap allocated data, undefined behavior, crashes and DoS.

7.1. CWE-787 - Out-of-bounds Write

CWE-787 vulnerabilities can be caused by *no/incomplete/invalid validation of input parameters* and include *wrong types* (e.g. CVE-2021-37650 - implementation assumes that all records in the examined dataset are strings), *wrong values* (e.g. CVE-2021-29571 - implementation assumes using correct values whereas an incorrect value can be passed) and *empty variables* as arguments (e.g. CVE-2021-29535 - it is assumed that input arguments are valid scalars and their numeric values are accessed directly. In the case of empty tensors, accessing the index 0 results in overflow). Another source is *no/incomplete/invalid validation of the values calculated inside the function*, e.g. CVE-2021-29540 - the computed size of the filter is not validated and used instead of the number of filter elements during convolutional filter backpropagation, which can result in an overflow when the computed size is too large. Another source can be *no validation for empty data structures or no validation whether the examined data structure has enough elements* for some operation (e.g. backpropagation) - e.g. CVE-2021-29578 - it is not checked whether the pooling sequence arguments have enough elements for the required shape. Also, when more than one tensor is used in the operation, the source can be *lack of shape compatibility validation of the tensors* (e.g. CVE-2021-29609 - it is not validated whether the second dimensions of sparse tensors used in the operation match). Last but not least, the issue can be connected to *no/improper validation of ML models* and specially crafted models can cause the vulnerability exploitation (e.g. CVE-2021-29603 - for TFLite).

How to exploit? - specially crafted inputs, models, - using function arguments with wrong values, wrong types and empty variables, - using empty structures or the incompatible ones for the operation (both one- and multi-tensor ones)

7.2. CWE-20 - Improper Input Validation

Main sources of CWE-20 vulnerabilities are *no/improper/insufficient checks whether examined data structures are not empty or have a proper dimension (shape)* and lack of handling of these occurrences (e.g. CVE-2020-15190 based on the boolean value, one of the tensors is the input tensor and the other one should be empty. The eager runtime traverses all output tensors, hence when the second tensor is examined, a reference to 'nullptr' is bind. Vulnerabilities can occur, because sometimes the implementation depends on the default values of axes, some quantities of the examined data structures are used to create different structures (it should be validated whether the original data structures shapes are proper for the new ones - e.g. CVE-2020-15199 - no validation whether the examined tensor has the minimum number of elements - required for the operation. This quantity is used to initialize a different data structure, which needs at least one element to not be 'nullptr'). The improper shape of one structure can entail the risk of another structure becoming a null pointer. Another issue can be that all assumptions of a method are not satisfied (e.g. CVE-2021-37674 - input tensors in this particular method have to be 4-dimensional, but it is not checked). Also, some external methods can be not prepared to operate with empty data structures - e.g. CVE-2020-26270 - running an LSTM/GRU model where the LSTM/GRU layer receives an input with results with 0 elements. The result is a CHECK failure when CUDA backend is used.

How to exploit? - wrong values of parameters and shapes of data structures, - empty structures, - using null pointers

7.3. CWE-190 - Integer Overflow or Wraparound

One source of CWE-190 vulnerabilities can be *using implementations and unsafe functions*, e.g. "InitDims" instead of "AddDimWithStatus" (or "BuildTensorShapeBase" in a tensor constructor), which can result in 'CHECK'-failures in the presence of overflows. This function is still used by some APIs, e.g. in file mkl_util.h (tensorflow/tensorflow/core/util/) - it is a potential vulnerability. Other source can be *no/improper validation of tensors' and models' sizes* - simple operations (e.g. concatenation - CVE-2021-29601) used with large models in TFLite can cause overflows (even more frequently than in traditional TensorFlow, as in this implementation int32 instead of int64 is often used). The example can be CVE-2021-29605 (TFLite code used for array allocation is vulnerable to an integer overflow - a special large model can be crafted such that the 'size' multiplier returns the value that overflows the "int" datatype and becomes negative. This negative value is given to "malloc" and as a result an invalid pointer is dereferenced. The other example can be CVE-2021-41198 - the number of elements in the output tensor is too large for "int64_t", overflow occurs and is detected via a "CHECK" statement (and process is aborted).

How to exploit? - using large inputs and demanding large outputs, - using specific APIs that still use 'AddDim' instead of 'AddDimWithStatus', - using specially crafted (also large) models, - building tensors without dimension bounds

7.4. CWE-119 - Improper Restriction of Operations within the Bounds of a Memory Buffer

Main sources of CWE-119 vulnerabilities are *no/incomplete/improper validation of input parameters of a function used to ac-*

cess and process the data structures (CVE-2020-15205 - one of the arguments lacks validation, thus values can be passed that can cause heap overflow/leak of contents of memory, which can contain return addresses and can be used to defeat the Address Space Layout Randomization - ASLR). Other issue can be *no/incomplete/improper validation of values calculated to use for accessing and processing the data* (also when validation is implemented for the debug mode, but not for the release mode). Here, problems related to *mimicking Python's indexing with negative values* can be considered - e.g. CVE-2020-15207 - to mimic Python's indexing with negative values, TFLite converts negative values to positive ones. The only check of the converted index is present in the debug builds. In the release mode, code execution continues with a negative index. Also, *no/incomplete/improper validation of indexes obtained using double indexing pattern* can be problematic (e.g. accessing the values of gradients in the reversed order - CVE-2020-15195). The next issue is *no/incomplete/improper validation of shapes of examined tensors*. Issues often occur when data structures are processed in parallel - (e.g. when more than one tensor is used in the operation and their shapes have to be equal and fully specified - e.g. CVE-2020-15198 - no validation whether the tensor with weights has the same shape as the data. Weights are accessed in parallel with the data, and since there is no validation, passing fewer weights than the data values can cause an out-of-bound read). Another problem is *no validation whether values given as parameters are not too large for the types used in the function and for memory allocation* - e.g. CVE-2020-15266 - when one of the arguments is very large, the CPU kernel implementation reads it as a C++ 'nan' floating point value. Operation on this variable can cause an undefined behavior and a segfault. The last issue found in the analysis regards *no/incomplete/improper checks whether the initialized values are created properly*.

How to exploit? - using large values of parameters, - using values (as indexes) not in the proper range of data, both positive and negative ones (also using operations based on reverse indexing), - using data structures with mismatched shapes when these structures are processed in parallel

7.5. CWE-125 - Out-of-bounds read

CWE-125 vulnerabilities can be caused by *no/incomplete/improper validation of user given parameters*. These issues can regard to *axis values* (e.g. CVE-2020-15265 - invalid axis value can be passed, resulting in accessing a dimension outside the tensor rank), *no checks of validity of strings* (e.g. representing format - see CVE-2020-26267), *no tensor bounds checks for tensor operations* (negative numbers, numbers exceeding the size), *no checks whether parameters used for a specific data structure specification are correct for its type* (e.g. CVE-2021-29559 - implementation assumes that the given parameters specify a valid sparse tensor). Other issues regard *no/incomplete/improper validation of user given data structures* (usually arrays): *empty checks* (e.g. CVE-2021-41219 - sparse matrix multiplication was vulnerable to binding a reference to "nullptr" when tensors with dimensions 0 or less were used). The fix has been introduced in the newest version of TensorFlow - 2.7.0. Also, *no/incomplete/improper validation of compatibility of data structures* can introduce vulnerabilities when they are all used together in the operation (also in parallel). Incompatibility can often result in binding a reference to a null pointer (e.g. CVE-2021-37655). Also, *very large values of parameters and tensors* can cause integer overflows resulting in an out-of-bounds read and disclosure of the process memory contents - CVE-2018-21233. The problem can be caused by a *missing check of correctness of array ranks*. Issues can be caused by *no control regarding incrementation of pointers* and trying to access data (e.g. CVE-2021-29532) or *using one variable*

for indexing two separate arrays in parallel and also using the debug checks (e.g. CVE-2020-15208 - when determining the common dimension size of two examined tensors, "DCHECK" is used - no-op outside of the debug mode. Attackers can use a larger tensor as the first argument to trigger the attack.) Also, *no/incomplete/improper validation of models* used in the operations can cause vulnerabilities (e.g. CVE-2021-29606 - a specially crafted TFLite model can cause OOB read). The last issue specified in the analysis is *improper control of the program flow* (e.g. CVE-2021-29551 - the implementation fails to terminate kernel execution if one validation condition fails and only returns from a function).

How to exploit? - specially crafted input parameters (e.g. shapes out of correct bounds), - using large data structures and values of parameters (also negative ones), - using empty structures as input parameters, - using tensors with different dimensions when the same dimension is a condition of a mathematical operation, - using incompatible values of parameters and input data structures in operations

7.6. CWE-476 - NULL pointer dereference

CWE-476 vulnerabilities are often caused by *problems with session* (no validation of availability of a session state, old TF versions do not set this state - e.g. CVE-2020-15204), *no/incorrect validation of tensor type* (conversion from a Python array to a C++ array is vulnerable to a type confusion - CVE-2021-29513) and *no/improper validation of tensor shape* (e.g. CVE-2021-41215 - the shape inference function assumes that the tensor has a positive rank and that the last dimension is correct). The problems can be introduced by *no/improper/incomplete validation whether input tensors are not empty or exist* (e.g. CVE-2021-41217 - the control flow graph building can cause null pointer exception when nodes cannot be paired). Problems can occur when *no/improper validation of input arguments in terms of their availability and correctness* is provided (e.g. CVE-2021-37639 - restoring tensors using raw APIs, TF can be tricked into dereferencing a null pointer if the tensor name is not provided). The next cause can be *no/improper validation of corner cases and all of the constraints of the operator* (e.g. CVE-2021-29616 - an undefined behavior can be triggered due to a dereference in corner cases that result in optimizing a no-input node). Yet another cause can be *improper control of the programme flow* (e.g. CVE-2021-37648 after unsuccessful validation kernel execution was not terminated, because it only exits a current function). Also, using unmaintained APIs can introduce vulnerabilities (e.g. CVE-2021-41208 - unmaintained boosted trees). The problem can be planned and not implemented validation (e.g. CVE-2021-29565 - missing validation under a "TODO").

How to exploit? - using tensors of non-numeric types, using empty tensors, - using parameters of the operation not satisfying the constraints of the operation (also corner cases), - using specially crafted models, - using one empty tensor and the other one/ones valid in multi-tensor operations, - using tensors with incorrect shape for the operation

8. Conclusions

While dedicated AI threats are undoubtedly extremely important and need to be addressed, we don't yet know to effectively combat them. That is why, in our study we focus on threats caused by standard C/C++ vulnerabilities in TensorFlow implementation. We analyze 6 common vulnerability types by summarizing meta-data regarding various vulnerabilities. We use Orthogonal Defect Classification, manual code analysis, and descriptions of specific CVEs to characterize their causes and results.

Though Tensorflow is a relatively new product, numerous security vulnerabilities regarding this framework have been found and potentially many more are still undetected. Some vulnerabilities considered in this study are very recent and they were only patched in the latest version of the product. The results show that the majority of TensorFlow vulnerabilities are not entirely different from issues regarding general-purpose products written in C/C++. However, they can be more difficult to find as it often requires deep semantic interpretation and good understanding of mathematical methods used. This seems to be the most likely cause of virtually zero effectiveness of examined SATs in detecting known vulnerabilities. Only one reported warning was included in one of the examined vulnerability fixes. Nevertheless, we find that some of the warnings can be used to improve the code quality and reduce the chance of introducing new issues. Potentially, some commercial SATs can be more effective (e.g. they can include more rules), however in our study we decided to use only freely available SATs that can be used by every researcher/programmer.

Since automated analysis is failing, it is up to developers to focus more on preventing the introduction and fixing existing vulnerabilities. Based on the results of our analysis in [Appendix A](#), we provide a checklist for developers/users of TensorFlow (and other numerical and DL libraries) that can be used to check their source code and potentially eliminate some common programming mistakes (we do not claim that these remarks are groundbreaking, however, the issues they relate to are still common and prominent). This list can also be used by programmers of non-AI platforms along with other secure programming guidelines, as the majority of the described issues also applies to general-purpose applications written in C/C++. We also believe that our checklist can be useful for the creators of new SATs (especially those that aim to be effective for the numerical libraries) - as a source of inspiration for new rules and to build the test cases. Here, we would like to encourage other researchers to use these insights to create tools that would aim to improve the security of numerical libraries - since this type of software is widely used and its security is becoming critical. It is vital to provide automatic tools that would be able to find at least some of the vulnerabilities in such products. We also aim to build such a tool in our future work based on the findings presented in the current paper. We would also like to encourage other researchers to examine different vulnerability types in TensorFlow (and possibly other numerical libraries) to describe even more thoroughly the threat landscape in these types of libraries.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Katarzyna Filus: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft, Writing – review & editing, Visualization. **Joanna Domańska:** Conceptualization, Validation, Formal analysis, Writing – review & editing.

Appendix A. Vulnerability Checklist

Based on the results of our analysis presented in the article, we provide a checklist for developers and users of TensorFlow library (and other numerical and DL libraries written especially in C/C++) that can be used to check their source code and potentially find and eliminate some common programming mistakes. This list can also be used by programmers of non-AI platforms along with other

guidelines - the majority of the issues also applies to general-purpose C/C++ applications. We also encourage creators of new static analysis tools to use this checklist as a source of inspiration for creating new rules and for building their test cases.

What can developers of DL libraries and TensorFlow do?

- use SATs to improve the quality of source code (be sure alerts are only false positives - e.g. memcpy usages) and use good programming practises to facilitate work of future programmers.
- cover important parts (e.g. validation) with code and not 'TODO's.
- **when dealing with C/C++ arrays (1):**
 - empty checks.
 - proper shape checks (dimensionality, number of necessary elements, they should be clearly described in the specification).
 - type checks.
 - checks for overflows regarding large data structures.
 - compatibility check when more than one array is used in an operation
 - check all the arrays - do not assume correctness by checking only some of them.
 - be careful when accessing arrays in parallel with one index - be sure that it is safe.
 - be careful when you use reverse indexing
- when dealing with pointers:
 - nullpointer checks.
 - when pointer is used to access array elements, check whether it is possible (e.g. pointer incrementation).
- **when dealing with variables in general (2):**
 - check whether they are assigned.
 - use type checks.
 - use format checks (e.g. when strings are given as an input).
 - use value checks
 - check limits for the type (to prevent overflows).
 - check assumptions of the method (they should be clearly described in the specification).
 - check compatibility with other arguments (e.g. when variable is used as an index).
- **do not use obsolete functions and replace them with safer alternatives (e.g. use 'AddDimWithStatus' instead of 'AddDim') (3).**
- do not shadow the variables used in the functions - this can make future developers' work more difficult and potentially lead to vulnerabilities.

What can developers of DL applications using TensorFlow do (or other similar libraries)?

For developers of external applications, the same advice as in (1) and (2) applies. They should validate the parameters and data structures passed to tensorflow. They do not have control over the implementation of tensorflow, e.g. over pointer incrementation or the use of obsolete functions - see (3), but in the second case they do have control over what APIs they use and here a risk assessment should be performed whether it is worthwhile to use an API that uses a particular function that may cause vulnerabilities.

Additionally, an extension of the above list can be introduced for developers of external applications using DL libraries:

- when it does not require to create very advanced method, better create your own solutions - this way you know what EXACTLY is implemented and even when the failure happens, you can easily correct them (no waiting for a 3rd party library fix required).

- use only maintained APIs, check for alternatives, when the desired API is unmaintained or create your own one.

References

- CWE Top 25 Most Dangerous Software Weaknesses, 2021. Available: https://cwe.mitre.org/top25/archive/2021/2021_cwe_top25.html [Accessed: 2021-11-29], [online].
- Abu Daoud, O., Albatayneh, O., Forslof, L., Ksaibati, K., 2021. Validating the practicality of utilising an image classifier developed using tensorflow framework in collecting corrugation data from gravel roads. *Int. J. Pavement Eng.* 1–12.
- Arusoaie, A., Ciobăca, S., Craciun, V., Gavrilut, D., Lucanu, D., 2017. A comparison of open-source static analysis tools for vulnerability detection in c/c++ code. In: 2017 19th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNAS). IEEE, pp. 161–168.
- Bridge, N., Miller, C., 1998. Orthogonal defect classification using defect data to improve software development. *Software Quality* 3 (1), 1–8.
- Chillarege, R., Bhandari, I.S., Chaar, J.K., Halliday, M.J., Moebus, D.S., Ray, B.K., Wong, M.-Y., 1992. Orthogonal defect classification-a concept for in-process measurements. *IEEE Trans. Software Eng.* 18 (11), 943–956.
- Commit fixing CVE-2020-26267 vulnerability. Available: <https://github.com/tensorflow/tensorflow/commit/ebc70b7a592420d3d2f359e4b1694c236b82c7ae> [Accessed: 2021-11-29], [online].
- Commit fixing CVE-2020-26270 vulnerability. Available: <https://github.com/tensorflow/tensorflow/commit/14755416e364f17fb1870882fa778c7fec7f16e3#L2864> [Accessed: 2021-11-29], [online].
- Commit fixing CVE-2021-29566 vulnerability. Available: <https://github.com/tensorflow/tensorflow/commit/3f6fe4def6f57e768260b48166c27d148f3015f> [Accessed: 2021-11-29], [online].
- Commit fixing CVE-2021-37637 vulnerability. Available: <https://github.com/tensorflow/tensorflow/commit/5dc7f6981fda74c8c5be41f393df705841fb7c5> [Accessed: 2021-11-29], [online].
- Commit fixing CVE-2021-37638 vulnerability. Available: <https://github.com/tensorflow/tensorflow/commit/301ae88b331d37a2a16159b65b255f4f9eb39314> [Accessed: 2021-11-29], [online].
- Committee, I.S.C., et al., 1990. IEEE standard glossary of software engineering terminology (ieee std 610.12–1990). los alamos, CA: IEEE Computer Society 169.
- Corallo, A., Lazoi, M., Lezzi, M., 2020. Cybersecurity in the context of industry 4.0: a structured classification of critical assets and business impacts. *Comput. Ind.* 114, 103165.
- CppCheck. Available: <https://cppcheck.sourceforge.io/> [Accessed: 2021-11-29], [online].
- Croft, R., Newlands, D., Chen, Z., Babar, M.A., 2021. An empirical study of rule-based and learning-based approaches for static application security testing. In: *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pp. 1–12.
- CVE - Common Vulnerabilities and Exposures. Available: <https://cve.mitre.org/> [Accessed: 2021-11-29], [online].
- CWE - Common Weakness Enumeration. Available: <https://cwe.mitre.org/index.html> [Accessed: 2021-11-29], [online].
- Fang, Y., Han, S., Huang, C., Wu, R., 2019. TAP: A static analysis model for PHP vulnerabilities based on token and deep learning technology. *PLoS ONE* 14 (11), e0225196.
- Filus, K., Boryszko, P., Domańska, J., Siavvas, M., Gelenbe, E., 2021. Efficient feature selection for static analysis vulnerability prediction. *Sensors* 21 (4), 1133.
- Filus, K., Siavvas, M., Domańska, J., Gelenbe, E., 2020. The random neural network as a bonding model for software vulnerability prediction. In: *Symposium on Modelling, Analysis, and Simulation of Computer and Telecommunication Systems*. Springer, pp. 102–116.
- FlawFinder. Available: <https://dwheeler.com/flawfinder/> [Accessed: 2021-11-29], [online].
- Chaffarian, S.M., Shahriari, H.R., 2017. Software Vulnerability Analysis and Discovery Using Machine-Learning and Data-Mining Techniques: A Survey. *ACM Computing Surveys (CSUR)* 50 (4), 1–36.
- Information technology Security techniques Information security management systems Overview and vocabulary. Standard, International Organization for Standardization, Geneva, CH, 2018.
- Kaur, A., Nayyar, R., 2020. A comparative study of static code analysis tools for vulnerability detection in c/c++ and java source code. *Procedia Comput. Sci.* 171, 2023–2029.
- Khaled, L., Abdelbaki, N., 2020. Evaluation of software static analyzers. In: *Proceedings of the 2020 9th International Conference on Software and Information Engineering (ICSIE)*, pp. 11–17.
- Kim, S., Woo, S., Lee, H., Oh, H., 2017. Vuddy: a scalable approach for vulnerable code clone discovery. In: 2017 IEEE Symposium on Security and Privacy (SP). IEEE, pp. 595–614.
- Kluban, M., Mannan, M., Youssef, A., 2022. On measuring vulnerable javascript functions in the wild. In: *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pp. 917–930.
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2021. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Trans Dependable Secure Comput.*
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y., 2018. Vuldeepecker: a deep learning-based system for vulnerability detection. *arXiv preprint arXiv: 1801.01681*.

- Medeiros, N., Ivaki, N., Costa, P., Vieira, M., 2020. Vulnerable code detection using software metrics and machine learning. *IEEE Access* 8, 219174–219198.
- Nong, Y., Cai, H., Ye, P., Li, L., Chen, F., 2021. Evaluating and comparing memory error vulnerability detectors. *Inf Softw Technol* 137, 106614.
- National Vulnerability Database (NVD). Available: <https://nvd.nist.gov/> [Accessed: 2021-12-02], [online].
- OWASP Secure Coding Practices Quick Reference Guide. Available: https://owasp.org/www-pdf-archive/OWASP_SCP_Quick_Reference_Guide_v1.pdf [Accessed: 2021-12-02], [online].
- Pang, B., Nijkamp, E., Wu, Y.N., 2020. Deep learning with tensorflow: a review. *Journal of Educational and Behavioral Statistics* 45 (2), 227–248.
- Pechenkin, A., Demidov, R., 2018. Applying deep learning and vector representation for software vulnerabilities detection. In: *Proceedings of the 11th International Conference on Security of Information and Networks*, pp. 1–6.
- Pereira, J.D., Campos, J.R., Vieira, M., 2021. Machine learning to combine static analysis alerts with software metrics to detect security vulnerabilities: An empirical study. In: *2021 17th European Dependable Computing Conference (EDCC)*. IEEE, pp. 1–8.
- Pereira, J.D., Ivaki, N., Vieira, M., 2021. Characterizing buffer overflow vulnerabilities in large c/c++ projects. *IEEE Access* 9, 142879–142892.
- Responsible AI practices Google AI. Available: <https://ai.google/responsibilities/responsible-ai-practices/?category=security> [Accessed: 2021-11-29], [online].
- Riom, T., Sawadogo, A., Allix, K., Bissyand'e, T.F., Moha, N., Klein, J., 2021. Revisiting the vccfinder approach for the identification of vulnerability-contributing commits. *Empirical Software Engineering* 26 (3), 1–30.
- Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., McConley, M., 2018. Automated vulnerability detection in source code using deep representation learning. In: *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, pp. 757–762.
- Samonas, S., Coss, D., 2014. The cia strikes back: redefining confidentiality, integrity and availability in security. *Journal of Information System Security* 10 (3).
- Shin, Y., Meneely, A., Williams, L., Osborne, J.A., 2010. Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities. *IEEE Trans. Software Eng.* 37 (6), 772–787.
- Shiraishi, S., Mohan, V., Marimuthu, H., 2015. Test suites for benchmarks of static analysis tools. In: *2015 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, pp. 12–15.
- Source Code Analysis Tools - OWASP. Available: https://owasp.org/www-community/Source_Code_Analysis_Tools [Accessed: 2022-09-26], [online].
- Source Code Security Analyzers - NIST. Available: <https://www.nist.gov/itl/ssd/software-quality-group/source-code-security-analyzers> [Accessed: 2022-09-26], [online].
- Steinhardt, J., Koh, P.W., Liang, P., 2017. Certified defenses for data poisoning attacks. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pp. 3520–3532.
- TensorFlow. Available: <https://www.tensorflow.org/> [Accessed: 2021-11-29], [online].
- Toloudis, D., Spanos, G., Angelis, L., 2016. Associating the severity of vulnerabilities with their description. In: *International Conference on Advanced Information Systems Engineering*. Springer, pp. 231–242.
- Veracode, 2020. State of Software Security Volume 11. Technical Report. Veracode.
- Visual Code Grepper. Available: <https://github.com/nccgroup/VCG> [Accessed: 2022-08-05], [online].
- Wei, H., Lin, G., Li, L., Jia, H., 2021. A context-aware neural embedding for function-level vulnerability detection. *Algorithms* 14 (11), 335.
- Xiao, Q., Li, K., Zhang, D., Xu, W., 2018. Security risks in deep learning implementations. In: *2018 IEEE Security and Privacy Workshops (SPW)*. IEEE, pp. 123–128.
- Zhao, J., Guo, S., Mu, D., 2021. DouBiGRU-A: Software defect detection algorithm based on attention mechanism and double BiGRU. *Computers & Security* 111, 102459.
- Zhioua, Z., Short, S., Roudier, Y., 2014. Static code analysis for software security verification: Problems and approaches. In: *2014 IEEE 38th International Computer Software and Applications Conference Workshops*. IEEE, pp. 102–109.

Katarzyna Filus works as a research assistant in the Institute of Theoretical and Applied Informatics, Polish Academy of Sciences, where she is approaching her PhD. She obtained her MSc degree from the Silesian University of Technology (Gliwice, Poland) in 2021. She took part in two projects under the EU H2020 program: SerIoT and SDK4ED. Her main areas of research include Machine Learning and Artificial Neural Networks and their application to several domains: network and software security, computer vision, signal analysis, localization and positioning.

Joanna Domańska received her Ph.D. in January 2005 from the Institute of Theoretical and Applied Informatics, Polish Academy of Sciences (ITAI PAS), and defended the habilitation dissertation in 2015 at the Silesian University of Technology. She is currently an Institute Professor at ITAI PAS. She took part in several projects for the National Science Centre (Poland) and in two projects under the EU H2020 program (SerIoT, SDK4ED). Her main areas of research include performance modelling, application of AI to networking research, improvement of QoS, energy-efficiency and security with Self-Aware Networks, IoT and software security.